Software Requirements Specification

for

PayMate

Version 0.1

Prepared by

Group Name: Thursday

Chayathon Chaiyasak	6522771300	6522771300@g.siit.tu.ac.th
Aekkarin Yimyaem	6522772498	6522772498@g.siit.tu.ac.th
Kitssada Pavasutthipand	6522772530	6522772530@g.siit.tu.ac.th
Nattapol Phanmadee	6522781994	6522781994@g.siit.tu.ac.th
Pattarapong Tiemtan	6522780897	6522780897@g.siit.tu.ac.th

Instructor: Asst.Prof.Dr. Sasiporn Usanavasin

Course: Software Engineering

Date: March 03, 2025

1 Introduction....3
1.1 Document Purpose....3
1.2 Product Scope....3
1.3 Intended Audience and Document Overview....4
1.4 Definitions, Acronyms and Abbreviations....5
1.5 Document Conventions....5
1.6 References and Acknowledgments....5
2 Overall Description....6
2.1 Product Overview....6
2.2 Product Functionality....6
2.3 Design and Implementation Constraints....7
2.4 Assumptions and Dependencies....9
3 Specific Requirements....10
3.1 Business and User Requirements....10
3.2 External Interface Requirements....11
3.3 Functional Requirements....11
3.4 Use Case Model....13
4 Other Non-functional Requirements....34
4.1 Performance Requirements....34
4.2 Safety and Security Requirements....34
4.3 Software Quality Attributes....35
5 Requirement Traceability....37
Appendix A – Data Dictionary....43

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
0.1	Nattapol Phanmadee, Aekkarin Yimyaem, Kitssada Pavasutthipand, Pattarapong Tiemtan, Chayathon Chaiyasak	First version - included basic design ideas, user requirements, functional requirements, quality requirements, and use case analysis of PayMate .	03/03/2025

1 Introduction

This Introduction section provides an overview of the Receipt Scanner and Expense Analyzer App, outlining its purpose, scope, and intended audience. It defines key terms, document conventions, and references while detailing the app's core functionalities, including receipt scanning, expense tracking, budget management, and report generation.

1.1 Document Purpose

This Software Requirements Specification (SRS) outlines the requirements for PayMate, a receipt scanner and expense analyzer application. The scope of this document covers the entire PayMate system, including its core functionalities, user management features, and integration capabilities. PayMate is designed to streamline personal finance management by providing users with tools to scan receipts, categorize expenses, track budgets, and optimize spending through cost-cutting suggestions. This SRS aims to detail all necessary software requirements to ensure that PayMate meets its intended functionality and user experience goals.

The scope of this document is comprehensive, covering both the front-end user interface and back-end processing capabilities of PayMate. It includes features such as bill splitting, receipt editing, automatic expense categorization, budget tracking, and integration with bank APIs. Additionally, it addresses user management aspects like login and registration, group sharing, and notification systems. This SRS serves as a guide for developers, stakeholders, and users to understand the full range of functionalities and expectations for the PayMate application. It does not focus on a specific subsystem but provides a holistic view of the entire application's requirements.

1.2 Product Scope

PayMate is a comprehensive receipt scanner and expense analyzer designed to simplify personal finance management. Its primary purpose is to help users efficiently track, categorize, and optimize their expenses by leveraging advanced technologies such as Optical Character Recognition (OCR) for receipt scanning and machine learning for expense categorization.

PayMate aims to provide users with a clear overview of their spending habits, enabling them to make informed financial decisions. The application offers features like bill splitting, budget tracking, and cost-cutting suggestions, making it an indispensable tool for individuals seeking to manage their finances effectively.

The benefits of using PayMate include enhanced financial transparency, reduced time spent on manual expense tracking, and improved budget adherence. By automatically categorizing expenses into predefined categories and providing insights into spending trends, PayMate helps users identify areas for cost reduction and optimize their financial resources. Additionally, features such as group expense logging and shared budget tracking facilitate collaborative financial management among friends or family members. Overall, PayMate's objectives are to streamline financial management, promote fiscal responsibility, and offer users a user-friendly interface to achieve their financial goals.

1.3 Intended Audience and Document Overview

PayMate is primarily intended for two key audiences:

1. **Client:** This document serves as a comprehensive representation of PayMate's intended functionality and requirements. The client can review this SRS to ensure the application aligns with their vision and expectations, verifying that all essential features and aspects are adequately addressed.
2. **Professor:** This SRS provides detailed information about the project's objectives, scope, and technical implementation. The professor can use this document to evaluate the project's adherence to software engineering principles and assess the students' understanding of requirements analysis and specification.

To facilitate a clear understanding of PayMate, this SRS is structured into the following sections:

1. **Introduction:** Provides an overview of the project, its purpose, and the intended audience of this document. It establishes the context for understanding the subsequent detailed requirements.
2. **Overall Description:** Describes the general context, functionality, constraints, and assumptions related to PayMate. This section offers a high-level perspective on the product and its interactions with the environment.
3. **Specific Requirements:** Details the specific business and user requirements, external interface requirements, functional requirements, and use case model for PayMate. This section is crucial for understanding the precise functionalities and user interactions supported by the system.
4. **Other Non-Functional Requirements:** Specifies the non-functional requirements, including performance, safety, security, and software quality attributes. This ensures that PayMate meets the necessary standards for usability, reliability, and security.
5. **Requirements Traceability:** This will link user requirements to test cases.
6. **Appendix A – Data Dictionary:** Provides a comprehensive list of data elements used within PayMate, along with their definitions, types, and constraints.
7. **Appendix B - Group Log:** Records the activities and contributions of the team members during the development of this SRS.

1.4 Definitions, Acronyms and Abbreviations

Term/Acronym	Description
API	Application Programming Interface
COMET	Concurrent Object-Oriented Modeling and Architectural Design
OCR	Optical Character Recognition
SRS	Software Requirements Specification
FR	Functional Requirement
TLS	Transport Layer Security
FR	Functional Requirement
UML	Unified Modeling Language

1.5 Document Conventions

Formatting Conventions:

- *Arial font, size 11 for regular text.*
- *Italics for comments and explanations.*
- *Bold for titles and important terms.*

Naming Conventions:

- *Functional Requirements: FR001, FR002, etc.*
- *Use Cases: UC001, UC002, etc.*
- *User Requirements: USR001, USR002, etc.*

1.6 References and Acknowledgments

SRS-template(interface style guide). Available at:
[https://docs.google.com/document/d/1EeawJ8HxI6IkovzOxO_otUemuDLPdnwC6Ry7dNR2JPw/edit?usp=sharing] (Accessed: 24 February 2025).

2 Overall Description

2.1 Product Overview

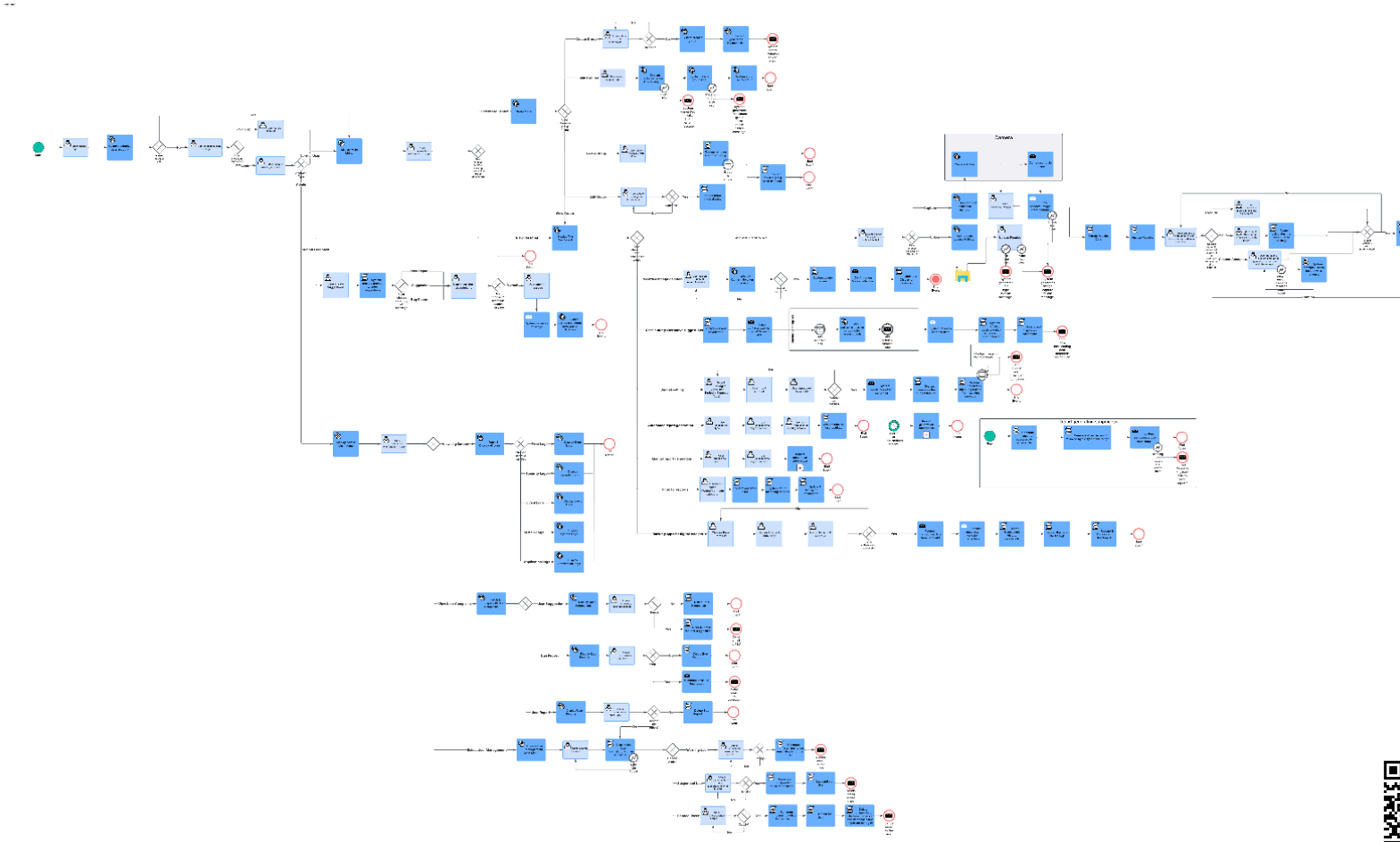
The receipt scanner and expense analyzer app is a self-contained product designed to help individuals and groups efficiently manage their receipts and track expenses. It is not a replacement for existing systems but aims to streamline the process of organizing, analyzing, and optimizing financial transactions. The app enables users to upload and categorize receipts, track spending, and manage budgets.

For personal use, the app allows users to input receipts either manually or by scanning them, and then automatically categorizes these expenses into predefined or user-defined categories. In addition, the app provides users with the functionality to set and manage budgets, ensuring that they stay within their financial limits. Users can also search for specific receipts or expenses and receive cost-cutting suggestions based on their spending patterns, such as finding cheaper alternatives for regularly purchased items.

For group use, the app allows users to create groups to share and manage expenses collaboratively. Group members receive notifications about new receipts or changes in expenses. The app also enables the generation of detailed expense reports, which can be exported in various formats (such as PDF or CSV), providing both personal and group users with a comprehensive view of their spending.

This app fits into a larger ecosystem that integrates with mobile devices for receipt scanning and cloud storage services for data backup and multi-device synchronization. It ensures that users can access and manage their financial information consistently across platforms, with cloud services safeguarding data integrity. This comprehensive feature set positions the app as a practical tool for individuals and groups to track, optimize, and analyze their expenses effectively.

The following is the BPMN diagram for the high-level process over



https://drive.google.com/file/d/1DuIOqlrRunrp0fuKGmsr0I149Eap4bVn/view?usp=sharing

2.2 Product Functionality

The app offers a comprehensive suite of features designed to help users track their receipts, analyze their spending, and optimize their finances. Whether for personal or group use, the app simplifies financial management by combining receipt scanning, budget management, expense reporting, and cost-cutting suggestions all in one platform.

Receipt Scanning & Input:

- Users can scan receipts using their device's camera or manually input details. The app automatically categorizes receipts based on predefined categories or custom user settings.

Expense Categorization:

- After receipt input, expenses are categorized (e.g., groceries, entertainment, transportation), allowing users to track their spending habits across different areas.

Cost-Cutting Suggestions:

- Based on spending patterns, the app analyzes user expenses and provides suggestions for cheaper alternatives for regularly purchased items.

Search Functionality:

- Users can search for specific receipts or expenses by category, date, or keywords, helping them quickly find and review particular transactions.

Budget Management:

- Users can set personal budgets for specific categories (e.g., groceries, dining) and track their spending progress to ensure they stay within their financial limits. Alerts are triggered when users approach or exceed their budget.

Group Expense Management:

- Users can create expense-sharing groups, allowing multiple users to input and track shared expenses. Group members can receive notifications about new entries or changes in the group's expenses.

Expense Report Generation:

- The app generates detailed expense reports summarizing a user's spending over time. These reports can be exported in formats like PDF or CSV for easy sharing and further analysis.

Multi-Device Synchronization:

- The app syncs data with cloud storage, allowing users to access their receipts, budgets, and reports across multiple devices, ensuring consistent information wherever they go.

2.3 Design and Implementation Constraints

Several constraints will influence the development of the receipt scanner and expense analyzer app. These constraints include hardware limitations, software tools, technologies, and design standards that must be adhered to during the development process.

Hardware Limitations:

- The app will need to perform receipt scanning using the camera on mobile devices. As such, it must be optimized for performance to handle image processing without excessive lag or memory usage. It should also be compatible with both Android and iOS devices, with support for varying camera resolutions and performance capabilities.

Performance Requirements:

- Given that users may input a high volume of receipts and transaction data, the app must be able to handle large datasets efficiently. Memory management and optimization for quick access to stored receipts and reports are essential. Real-time notifications for group use will also require efficient background operations.

Technology Stack:

- The app will be developed using native mobile technologies, such as Swift for iOS and Kotlin for Android, to ensure high performance and a responsive user interface. The cloud storage service for synchronization will be built using Firebase or AWS, providing secure, reliable storage and real-time synchronization.
- The COMET method will be used for the software design, ensuring that we follow an iterative, modular approach to development. This method will help us manage complexity and reduce the risk of errors during development.
- UML (Unified Modeling Language) will be utilized for designing the system architecture, including class diagrams, use case diagrams, and sequence diagrams, to ensure clarity and consistency in the system's design.

Interfaces to Other Applications:

- The app will need to integrate with cloud storage platforms for syncing data across devices. Additionally, users may wish to export reports to other tools (e.g., Excel or accounting software). Therefore, it will be essential to implement export functionality in formats like CSV or PDF.

Security Considerations:

- As the app deals with sensitive financial data, robust security measures will be implemented. This includes end-to-end encryption for receipt data, secure authentication

methods (such as OAuth2 for group accounts), and compliance with data privacy regulations (such as GDPR or CCPA).
- Regular security updates and patches will be needed, and all data stored in the cloud will be encrypted at rest and during transmission.

Communication Protocols:

- Communication between the app and the cloud services (for data syncing, receipt upload, and report generation) will use HTTPS to ensure secure transmission. Additionally, real-time communication for notifications will rely on WebSocket or Firebase Cloud Messaging (FCM).

Design Conventions & Programming Standards:

- The development team must follow standard mobile app development practices, including using MVC (Model-View-Controller) for app architecture and MVVM (Model-View-ViewModel) for managing data and UI interactions.
- Code readability and documentation standards will be emphasized to ensure maintainability, especially since the app will require future updates and bug fixes.
- The app must comply with accessibility standards, ensuring that it is usable by people with disabilities. This includes implementing support for screen readers and providing high contrast visuals.

By adhering to these constraints and standards, the development of the receipt scanner and expense analyzer app will be streamlined, secure, and capable of delivering a high-quality user experience across different platforms.

2.4 Assumptions and Dependencies

Assumptions:

1. **Users Have Access to the Feedback System** – It is assumed that users can navigate to the feedback submission section through the mobile application without restrictions.
2. **Internet Connectivity is Stable** – The feedback process requires an active internet connection for successful submission. If the connection is lost, users may need to retry.
3. **Feedback Processing is Automated** – The system automatically stores and processes the feedback without requiring manual intervention unless flagged for moderation.
4. **User Authentication is Functional** – If the feedback system requires users to log in, it is assumed that authentication services are operational.
5. **Data Storage and Retrieval are Reliable** – The system correctly stores user feedback in the database and allows administrators to access and review it when needed.

Dependencies:

1. **Database System** – The feedback system depends on a functional database to store and retrieve feedback data.
2. **Notification Service** – If users receive confirmation messages (email), the system depends on external notification services.

3. **Content Moderation Service** – If automated moderation is required, the system may depend on third-party moderation tools or AI-based filtering.
4. **Third-Party API Integrations** – If feedback analytics or sentiment analysis is involved, the system may rely on external APIs for processing.
5. **Hosting and Server Availability** – The feedback system depends on a web server or cloud service for operation. Downtime could impact submission and processing.

3 Specific Requirements

3.1 Business and User Requirements

3.1.1 Business Objectives

Develop an all-in-one, OCR-powered financial management platform designed to automatically capture and classify expenses from both paper and digital receipts. The system will enable customizable bill itemization, equitable cost splitting, and cost-cutting suggestions to promote smarter spending habits. Users can set and monitor personal or group budgets, generate time-customizable periodic reports, and integrate with banking apps for seamless digital receipts.

To ensure data privacy and integrity, all sensitive information will be encrypted using AES-256, and robust user account management (registration, login, and secure sessions) will be implemented. Administrators will have access to real-time system logs, user profiles, and the ability to suspend/unsuspend accounts, helping maintain a safe and compliant environment. By offering a user-friendly dashboard that consolidates spending trends, payment details, and feedback channels, this project aims to deliver a transparent, efficient, and collaborative money-tracking experience suitable for individuals and group households alike

3.1.2 User Requirements (User Stories)

USR001: User want to customise the bills (ex. choose certain items in the bill)
USR002: User want to equally split the bill.
USR003: User wants cost-cutting alternative suggestions
USR004: User wants detailed record (ex. payment method, store)
USR005: User wants a dashboard containing a brief summary of all info
(e.g. total expense of the month, expense trends)
USR006: User wants integration with banking apps for digital receipts
USR007: User wants a budget setting function
USR008: User wants a log of past receipts
USR009: User wants time-customizable periodic expense reports
USR010: User wants to upload/scan pictures of payment receipts
USR011: User wants to track expense as a household unit/group of users
USR012: User wants to register and login to the system
USR013. User wants to submit various types of feedback
USR014: Admin wants to be able to view user profile and activities
USR015: Admin wants real-time system logs
USR016: Admin wants to be able to suspend/unsuspend user accounts
USR017:. Admin wants to view user suggestion

3.2 External Interface Requirements

3.2.1 Hardware Interfaces

PayMate interacts with the following hardware components:

Mobile Device Camera: The mobile device's camera is used to capture images of receipts for data extraction using OCR technology. The application requires access to the camera to function.

Touchscreen Display: The touchscreen display of the mobile device provides the primary interface for user interaction with the PayMate application. The application supports standard touch gestures for navigation and data input.

3.2.2 Software Interfaces

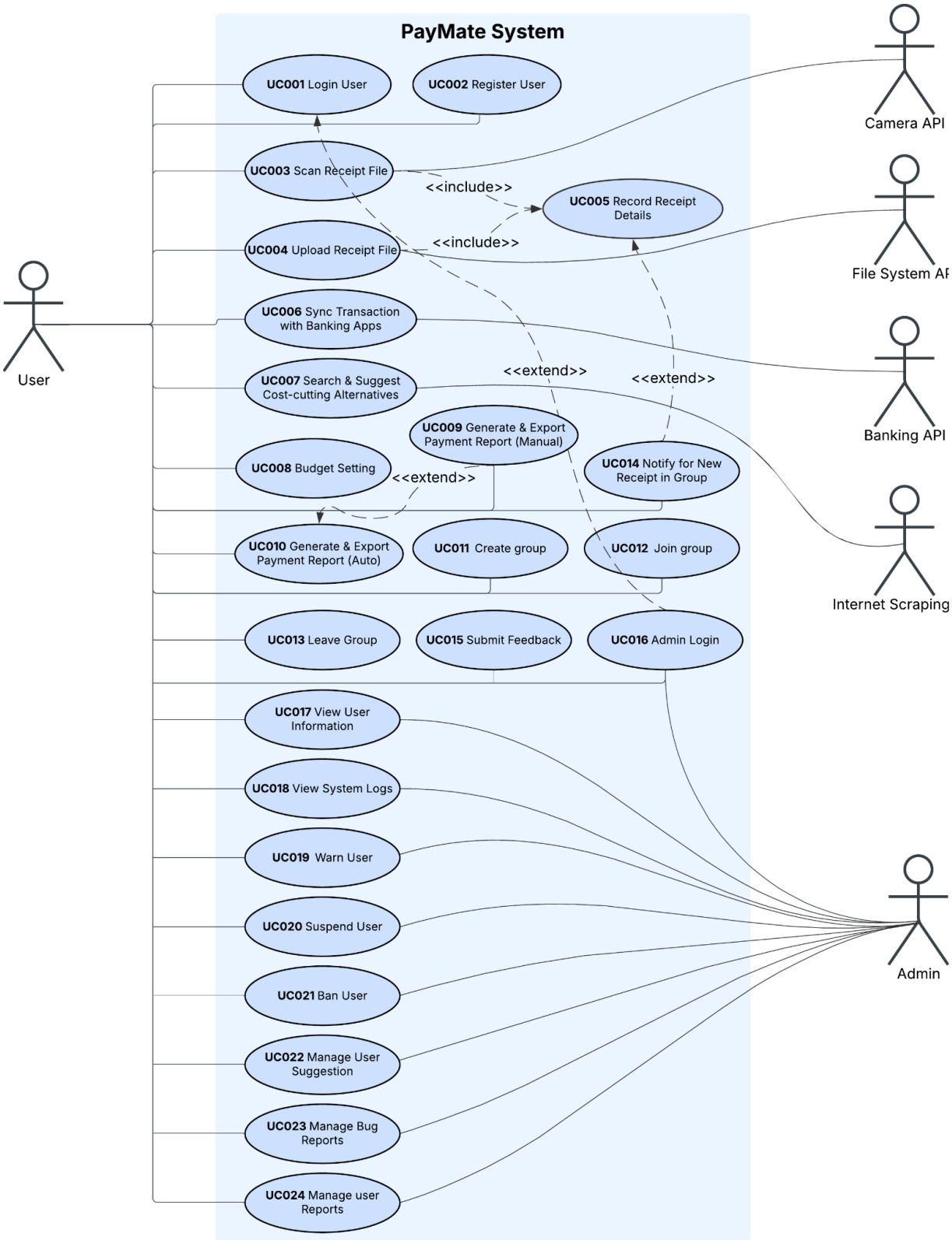
Bank APIs: Interfaces with third-party APIs to synchronize transaction data.

Web Services: Interfaces with web services to fetch cost-cutting alternatives and generate report

3.3 Functional Requirements

FR001: The system shall allow the user to submit feedback on budget distribution and automation.
FR002: The system shall allow the user to select between "splitting" (dividing the total amount equally among a specified number of people) or "custom amount" (allocating a set amount to certain users for the transaction).
FR003: The system shall automatically extract items, store name, payment method, date, and site from different types of receipts.
FR004: The system shall automatically categorize expenses into predefined categories (e.g., Groceries, Utilities, Rent, Entertainment).
FR005: The system shall provide suggestions and comparisons for cost-effective alternatives.
FR006: The system shall allow users to integrate with bank APIs for sync transactions.
FR007: The system shall allow users to set monthly spending limits per category.
FR008: The system shall generate reports for custom time spans of 1 month, 6 months, and 1 year.
FR009: The system shall allow the user to upload receipts via file upload.
FR010: The system shall allow users to manually enter expenses.
FR011: The system shall allow users to edit and delete expense entries.
FR012: The system shall allow users to search and filter expenses by different parameters (e.g., date, category, store).
FR013: The system shall allow users to add notes and tags to transactions.
FR014: The system shall allow users to share expenses and reports with other users.
FR015: The system shall allow the user to register as a new user.
FR016: The system shall allow the user to create and submit general suggestions.
FR017: The system shall allow the user to add multiple bank accounts.
FR018: The system shall allow the user to sync with credit cards, bank transactions, and subscription payments.
FR019: The system shall allow the system admin to manage users, categories, and settings.
FR020: The system shall allow users to fetch user suggestions, budget reports, and cost methods.
FR021: The system shall allow the admin to log in using existing credentials.

3.4 Use Case Model



3.4.1 Use Case #1 UC001 Log-in User

Author – Nattapol Phanmadee

Purpose - To log user into the system

Requirements Traceability – User Email, User Password

Priority - High

Preconditions - Nothing

Post conditions - User is logged in with a valid session ID token

Actors – User Select "Login" button on the screen

Flow of Events

1. User opens the app.
2. System displays log-in/registration menu.
3. User enters log-in credentials.
4. System verifies user's credentials.
5. If user exists, system generates a session ID token. Otherwise, go to (A)
6. System sends user to landing page.
7. System logs the user's entry.

Exceptional flow - (A) The system display error message and prompt the user to re-enter both information.

3.4.2 Use Case #2 UC002 Register User

Author – Nattapol Phanmadee

Purpose - To register user into the system

Requirements Traceability – User and PayMate System

Priority - High

Preconditions - Nothing

Post conditions - User is logged in with a valid session ID token

Actors – User select "Login" button on the screen and Paymate will interact with the Email API

Flow of Events

1. User opens the app.
2. System displays log-in/registration menu
3. User enters log-in credent
4. System verifies user's credentials.
5. If user exists, system generates a session ID token. Otherwise, go to (A)
6. System sends user to landing page.
7. System logs the user's entry.

Exceptional flow - (A) The system display error message and prompt the user to re-enter both information.

3.4.3 Use Case #3 UC003 Scan Receipt File

Author – Nattapol Phanmadee

Purpose - Allow user to scan receipts into the system.

Requirements Traceability – Camera Input

Priority - Medium

Preconditions - User is logged in with a valid session ID token, and the app is allowed to use the camera.

Post conditions - System temporarily stores OCR-scanned text.

Actors – User select "Input Receipt" button on the screen, then press "Capture" then Camera will be activate.

Flow of Events

1. User selects "Input Receipt" button.
2. User selects "Capture" option.
3. System contacts Camera API
4. Camera deploys
5. User captures image with Camera
6. Camera sends image to system.
7. System receives image from camera.

8. System uses OCR to scan text from image file.
9. If system detects text in image file, store it temporarily. Otherwise, go to (A)

Include- UC005

Exceptional flow - (A) System sends an error message and prompts the user to capture a new image.

3.4.4 Use Case #4 UC004 Upload Receipt File

Author – Nattapol Phanmadee

Purpose - Allow user to upload receipts into the system.

Requirements Traceability – Camera Input

Priority - Medium

Preconditions - User is logged in with a valid session ID token, and the app is allowed to access the folder.

Post conditions - System temporarily stores OCR-scanned text.

Actors – Select "Input Receipt" button on the screen, then press "Upload". PayMate system will lead to file system.

Flow of Events

1. User selects "Input Receipt" button.
2. User selects "Upload" option.
3. System contacts Folder API.
4. File System deploys.
5. User selects file to upload.
6. Camera sends image to system.
7. System receives file from camera.
8. If file type is valid, continue. Else, go to (A)
9. System uses OCR to scan text from image file.
10. If system detects text in image file, store it temporarily. Otherwise, go to (B)

Include-UC005

Exceptional flow - (A) System sends an error message and prompts the user to upload file of a valid type.

(B) System sends an error message and prompts the user to capture a new image.

3.4.5 Use Case #5 UC005 Extract Receipt Details

Author – Nattapol Phanmadee

Purpose - To extract receipt details for further processing.

Requirements Traceability – OCR-scanned text.

Priority - Medium

Preconditions - System temporarily stores OCR-scanned text.

Post conditions - System temporarily stores receipt items data.

Actors – System successfully extracts text from a given file using OCR.

Flow of Events

1. System retrieves OCR-scanned text
2. System analyzes text to find receipt items, store name, payment method, date, and time.
3. System assigns a category to each item.
4. If system successfully detects items, continue. Otherwise, go to (A)
5. System displays list of extracted items.
6. User optionally edits info extracted item & assigned category. During this, user may choose to abort process, in which case go to (B)
7. User confirms extracted item list.
8. System records receipt & extracted items.
9. System updates the dashboard with new data.

Exceptional flow - (A) System sends an error message, and prompts the user to input a new file.

(B) System prompts the user to select 'Yes' or 'No' to guarantee aborting process. If yes, system aborts process and returns to dashboard page. Else, return flow to 5.

3.4.6 Use Case #6 UC006 Sync Transaction with Banking Apps

Author – Chayathon Chaiyasak

Purpose - Sync Transaction with Banking Apps.

Requirements Traceability – Nothing

Priority - Low

Preconditions - User is logged in with a valid session ID token

Post conditions - User get transaction from banking apps

Actors – User select "Banking transaction sync" button on the screen ,then the PayMate will request data from banking app.

Flow of Events

1. User click "Dashboard" button.
2. System display dashboard's of user.
3. User click "Banking transaction sync" button
4. System request to access the transaction log of banking app.
5. Banking apps accept request from Paymate.
6. User enter their bank password
7. Banking apps verifies user's password. Otherwise, go to (A)
8. Banking apps transfer data to Paymate.

Exceptional flow - (A) The banking apps will let you try to enter 4 more time to re-enter password again.

3.4.7 Use Case #7 UC007 Search & Suggest Cost-cutting Alternatives

Author – Chayathon Chaiyasak

Purpose - Suggest cost-cutting ways for different user

Requirements Traceability – Record of user’s data

Priority - Low

Preconditions - User have account and store records before

Post conditions - User get different suggestion on cost-cutting for different user and record

Actors – User select "Cost-cutting" button on the screen, then PayMate contacts the external Internet Scraping API.

Flow of Events

1. User click "Dashboard" button.
2. System display dashboard's of user.
3. User click "Cost-cutting" button.
4. System analyse user's record.
5. System contacts external Internet Scraping API.
6. API performs internet scraping for relevant deals.
7. API send data to PayMate.
8. System receives data.
9. System analyze deals.
10. Update dashboard.

3.4.8 Use Case #8 UC008 Budget Setting

Author – Kitssada Pavasutthipand

Purpose - To allow user to set and track their budget effectively

Requirements Traceability – Budget Type (Time Period,Expense Type),Budget Amount, Alert Threshold

Priority - Low

Preconditions - User is logged in with a valid session ID token

Post conditions - A new (or updated) budget is saved. The system continuously tracks expenses and notifies the user if usage exceeds the threshold

Actors – User select "Manage Budget" option in dashboard

Flow of Events

1. User select "Budget Setting" option in dashboard

2. System displays budget setting option
3. User selects Budget Type (Time Period + Expense Type)
4. System prompts user to input budget amount
5. User inputs budget amount
6. System confirm and asks for an alert threshold
7. User inputs an alert threshold
8. System validates user input, if any required data is missing or invalid, go to (A)
9. User confirmation
10. If user confirms, system saves the budget settings and begins monitoring expenses. Otherwise, go to (B)
11. System calculates budget usage from the existing receipt
12. If usage exceeds threshold, system sends “Budget Over Threshold” notification.

Exceptional flow - (A) If required data is missing or invalid, the system displays an error message and prompts the user to correct the input

(B) If user is not satisfied with the setting, the system allows user to re-enter or adjust budget setting

3.4.9 Use Case #9 UC009 Generate & Export Payment Report (Manual)

Author – Aekkarin Yimyaem

Purpose - To manually generate and export payment reports

Requirements Traceability – Date range, Report type

Priority - Medium

Preconditions - User is logged into the PayMate system.

Post conditions - Report is generated and exported in the specified format.

Actors – User selects "Generate Report"

Extends – UC010

Flow of Events

1. User selects the "Reports" option .
2. The system displays available report types and filtering options.
3. User selects the desired report type, date range.
4. The system generates the payment report based on the selected criteria.

5. User clicks the "Export" button.
6. The system exports the report in the specified format.
7. The system generate password to access file.
8. The system send the report via email using the Email API.(A)

3.4.10 Use Case #10 UC010 System: Generate & Export Payment Report (Auto)

Author – Aekkarin Yimyaem

Purpose - To automatically generate and export payment reports based on a schedule.

Requirements Traceability – Date range, Report type

Priority - Medium

Preconditions - User is logged into the PayMate system.

Post conditions - Report is generated and exported in the specified format.

Actors – User selects "Auto generate Report"

Flow of Events

1. The system schedule triggers report generation based on predefined settings.
2. The system generates the report based on the pre-defined criteria.
3. The system generate password to access file.
4. The system send the report via email using the Email API.(A)

Exceptional flow - (A) If sending the report via email fails, the system displays a warning message but still provides the report as a downloadable file.

3.4.11 Use Case #11 UC011 Create group

Author – Chayathon Chaiyasak

Purpose - Create group of user to share records and analysis statistic

Requirements Traceability – Group information

Priority - Medium

Preconditions - User have account

Post conditions - User create group and get a invitation link

Actors – User select "Create group" button on the screen

Flow of Events

1. User enters group menu.
2. System display group menu.
3. User click "Create group" button
4. User enter group information
5. System create group.
6. System create and send invitation link to user.

3.4.12 Use Case #12 UC012 Join group

Author – Chayathon Chaiyasak

Purpose - Join group of user to share records and analysis statistic

Requirements Traceability – Invitation link

Priority - Medium

Preconditions - User have account and invitation link

Post conditions - User join the group

Actors – User select "Join group" button on the screen

Flow of Events

1. User enters group menu.
2. System display group menu.
3. User click "Join group" button.
4. User enter invitation link.
5. System check invitation link validity. Otherwise, go to (A)
6. System check group limit. Otherwise, go to (B)
7. System adds user to group

Exceptional flow - (A) System generates 'invalid link' error message.

(B) System generates 'Maximum group size reach' error message.

3.4.13 Use Case #13 UC013 Leave Group

Author – Kitssada Pavasutthipand

Purpose - Allow the user to exit a group, removing their membership and associated privileges.

Requirements Traceability – User's confirmation to leave the group

Priority - Medium

Preconditions - User is logged in with a valid session ID token and user is currently a member of the group

Post conditions - The user is no longer listed as a member of the group. If other members exist, the user is simply removed from the membership list and loses access. If the user was the last member of the group, the group is deleted automatically.

Actors – User select a "Leave Group" option

Flow of Events

1. User navigates to the group setting page
2. System display the group details,including "Leave Group" option
3. User click "Leave Group"
4. System request confirmation. Otherwise, go to (A)
5. User confirm the decision to leave group
6. System checks the number of current members.
7. If other members exist, the user is removed from the group membership list.
8. If this user is the last member, the system deletes the group automatically.
9. System displays a success message (e.g., “You have left the group.”). Otherwise, go to (B)
10. System notifies remaining group member that the user has left

Exceptional flow - (A) If the user decides not to leave when prompted for confirmation, the system abort the process

(B) If the system cannot process the request, its display an error message and prompts the user to try again later

3.4.14 Use Case #14 UC014 Notify for New Receipt in Group

Author – Aekkarin Yimyaem

Purpose - Notify users in a group chat when a new receipt is available.

Requirements Traceability – Receipt details (amount, date, group chat ID)

Priority - Medium

Preconditions - User is part of the group.

Post conditions - New notification is inserted into group member's inbox.

Actors – User insert new receipt is created while in the group.

Extends – UC005

Flow of Events

1. A new receipt is created in the system.
2. The system composes a notification message containing receipt details.
3. The system sends the notification message to the identified group.

3.4.15 Use Case #15 UC015 Submit Feedback

Author – Kitssada Pavasutthipand

Purpose -Allow a user to provide suggestions or feedback to the system (e.g., new features, improvements, or general ideas).

Requirements Traceability – Suggestion Content (text, attachments, screenshots, etc.)

Priority - Medium

Preconditions - User is logged in with a valid session ID token,System can store or log suggestions

Post conditions - Suggestion is saved in the system,The user receives acknowledgment (email or in-app notification).Administrators can review or respond to the suggestion.

Actors –User decides to submit a suggestion (e.g., clicks “Send Suggestion” or “Feedback” in the application menu).

Flow of Events

1. User select "Submit feedback" option in dashboard.
2. System display a suggestion form
3. System display option of categorize message
4. User select categorize for message.
5. System save the selected categories for message
6. User fills in the form with details
7. User confirm to sent message. Otherwise, go to (A)
8. System performs basic validation. Otherwise, go to (B)
9. System saves the suggestion in it internal database based on categories. Otherwise, go to (C)
10. System sends a confirmation to the user
11. System notifies an administrator

Exceptional flow - (A) User abaddon the data: If user leave message without confirm or abandon message, the system will roll back to main menu of the system and delete the message.

(B) Missing Required Data: If the user leaves required fields empty or provides invalid data, the system displays an error message and prompts the user to correct the data before resubmitting.

(C) If the system cannot save the suggestion (e.g., database error, network failure), it displays an error notification and may instruct the user to try again later.

3.4.16 Use Case #16 UC016 Admin Login

Author – Pattarapong Tiemtan

Purpose - To log in into the admin system

Requirements Traceability – Admin Email, Admin Password

Priority - High

Preconditions - Nothing

Post conditions - Admin is logged in with a valid session ID token

Actors – Admin tap "Login" button on the screen

Extends – UC001

Flow of Events

1. Admin opens the app.
2. System displays log-in/registration menu.
3. Admin enters log in credentials.
4. System verifies admin's credentials.
5. If admin account exists, system generates a session ID token. Otherwise, go to (A)
6. System sends admin to the admin landing page.
7. System logs the admin's entry.

Exceptional flow - (A) The system display error message and prompt the user to re-enter both information.

3.4.17 Use Case #17 UC017 View User Information

Author – Pattarapong Tiemtan

Purpose - To view the logs of user information, profile, and activities

Requirements Traceability – User ID

Priority - High

Preconditions - Account type must be admin

Post conditions - Admin get the report of user profile, information, and activity logs

Actors – Admin tap search icon on the screen

Flow of Events

1. Admin opens the app.
2. System displays log-in/registration menu.
3. Admin enters log in credentials
4. System verifies admin's credentials.
5. If admin account exists, system generates a session ID token. Otherwise, go to (A)
6. System sends admin to the admin landing page.
7. System logs the admin's entry.
8. Admin select user management

9. System change page to user management page.
10. Admin search the user with user ID or username
11. System logs the searching
12. System fetch the user information if the user exists. Otherwise, go to (B)
13. System display the user information.

Exceptional flow - (A) The system display error message and prompt the user to re-enter both information.

(B) The system display error message and prompt the admin to re-enter the valid username or user ID.

3.4.18 Use Case #18 UC018 View System Logs

Author – Chayathon Chaiyasak

Purpose - To view the logs of error logs, security logs, event logs, system logs, and application logs

Requirements Traceability – Need to login as Admin

Priority - Medium

Preconditions - Any condition that must be satisfied before the use case begins

Post conditions - The conditions that will be satisfied after the use case successfully completes

Actors – Admin tap "System Logs" on the screen

Flow of Events

1. Admin select "System log" button.
2. System display all log.
3. Admin click "Sort" button.
4. System display different specific options of log (Error,Security, Event, System, Application)
5. Admin choose specific option.
6. System display specific options of log base on option admin choose.

3.4.19 Use Case #19 UC019 Warn User

Author – Pattarapong Tiemtan

Purpose - To send the warning message email to the user

Requirements Traceability – User ID, Warning Reason

Priority - Medium

Preconditions - Account type must be admin and the user must exist

Post conditions - The email sends to the user's email.

Actors – Admin tap confirm icon on the screen

Flow of Events

1. System verifies admin's credentials.
2. System change page to user management page.
3. Admin select "User Management" button.
4. Admin search the user with user ID or username.
5. System fetch the user information if the user exists. Otherwise, go to (A)
6. System display the user information.
7. Admin choose the "Warning User" button.
8. System change page to warning user page.
9. Admin select the reason of warning from the drop down menu.
10. Admin click "confirm" button
11. System generate the message with the detail from the input of admin.
12. System sends the message through the email to the user email.

Exceptional flow - (A) The system display error message and prompt the admin to re-enter the valid username or user ID.

3.4.20 Use Case #20 UC020 Suspend User

Author – Pattarapong Tiemtan

Purpose - To send the suspend message email to the user and suspend the user

Requirements Traceability – User ID, Suspend Reason, Suspend Length, Start Date

Priority - Medium

Preconditions - Account type must be admin and the user must exist

Post conditions - The conditions that will be satisfied after the use case successfully completes

Actors – Admin tap confirm icon on the screen

Flow of Events

1. System verifies admin's credentials.
2. System change page to user management page.
3. Admin select "User Management" button.
4. Admin search the user with user ID or username.
5. System fetches the user information if the user exists. Otherwise, go to (A)
6. System displays the user information.
7. Admin choose the "Suspend User" button.
8. System change page to suspend user page.
9. Admin select the reason of suspend from the drop down menu.
10. Admin select the suspend length and when to start.
11. Admin click "confirm" button
12. System generates the message with the detail from the input of the admin.
13. System suspends the user.
14. System sends the message through the email to the user.

Exceptional flow - (A) The system display error message and prompts the admin to re-enter the valid username or user ID.

3.4.21 Use Case #21 UC021 Ban User

Author – Pattarapong Tiemtan

Purpose - To send the ban message email to the user and ban the user

Requirements Traceability – User ID, Ban Reason, Start Date

Priority - Medium

Preconditions - Account type must be admin and the user must exist

Post conditions - The email sends to the user's email, delete user's information except username and ban user

Actors – Admin tap confirm icon on the screen

Flow of Events

1. System verifies admin's credentials.
2. System change page to user management page.
3. Admin select "User Management" button.
4. Admin search the user with user ID or username.
5. System fetches the user information if the user exists. Otherwise, go to (A)
6. System displays the user information.
7. Admin choose the "Ban User" button.
8. System change page to ban user page.
9. Admin select the reason of ban from the drop down menu.
10. Admin chooses when to start.
11. Admin click "confirm" button
12. System generates the message with the details from the input of the admin.
13. System ban the user.
14. System delete user information except username.
15. System sends the message through the email to the user.

Exceptional flow - (A) The system displays an error message and prompts the admin to re-enter the valid username or user ID.

3.4.22 Use Case #22 UC022 View User Suggestions

Author – Pattarapong Tiemtan

Purpose - To view the user suggestions and forward the suggestion to the related department

Requirements Traceability – Login as admin

Priority - Medium

Preconditions - Account type must be admin

Post conditions - View the suggestion and delete the suggestion or the suggestion email sends to the related department's email

Actors – Admin tap confirm icon on the screen

Flow of Events

1. System verifies admin's credentials.
2. Admin selects "User Complain" button.
3. System change page to user complain page.

4. Admin clicks "User Suggestion" button.
5. System fetch and display the user suggestions.
6. Admin chooses one user suggestion.
7. Admin chooses to delete or send the suggestion.
8. Admin selects "Confirm" button.
9. If "delete" is selected then remove the suggestion from the database, else generate the email with suggestion detail and sends to the related department.

3.4.23 Use Case #23 UC023 Bug Report

Author – Nattapol Phanmadee

Purpose - To view the bug report and forward the report to the related department

Requirements Traceability – Login as user

Priority - Medium

Preconditions - Account type must be admin

Post conditions - View the bug report and delete the report or the report email sends to the related department's email

Actors – User tap confirm icon on the screen

Flow of Events

1. System verifies admin's credentials.
2. Admin selects "User Complain" button.
3. System change page to user complain page.
4. Admin clicks "Bug Report" button.
5. System fetch and display the bug reports.
6. Admin chooses one bug report.
7. Admin chooses to delete or send the bug report.
8. Admin selects "Confirm" button.
9. If "delete" is selected then remove the bug report from the database, else generate the email with bug report detail and sends to the related department.

3.4.24 Use Case #24 UC0024 User Report

Author – Nattapol Phanmadee

Purpose - To view the user report

Requirements Traceability – Login as admin

Priority - Medium

Preconditions - Account type must be admin

Post conditions - View the user report or delete the report

Actors – User tap confirm icon on the screen

Flow of Events

1. System verifies admin's credentials.
2. Admin selects "User Complain" button.
3. System changes page to user complain page.
4. Admin clicks "User Report" button.
5. System fetches and displays the user reports.
6. Admin chooses one bug report.
7. Admin chooses to delete or accept the user report.
8. Admin selects "Confirm" button.
9. If "delete" is selected then remove the user report from the database, else
10. save the user detail and send to the user management process. System changes page to user management page.

4 Other Non-functional Requirements

4.1 Performance Requirements

P1. The customise bill selection response time should be within 1 second
P2. Splitting bill into equal parts should be completed within 2 seconds
P3. Data extraction of a detailed record should be completed within 3 seconds
P4. Data categorization should be completed within 1 second
P5. Notification shall be delivered within 5 seconds of new-cost-cutting suggestion being found
P6. The dashboard should load the expense summary in under 2 seconds for optimal user experience
P7. Trend calculations should be complete in 3 seconds for datasets under 12 months
P8. Sorting operation should complete within 1 second for datasets under 1000 receipts
P9.The report generation process should complete within 5 seconds for a 1-month timeframe, 10 seconds for a 6-month timeframe, and 15 seconds for a 1-year timeframe.
P10. Automated reports should be generated within 1 minute of scheduled time.
P11. OCR processing speed is 5 seconds maximum.
P12. Item extraction should complete in under 5 seconds per receipt.
P13. Item selection and update should be processed within 1 second.
P14. Links should be valid for 24 hours and regenerated regularly.
P15. The login process should take no longer than 2 seconds
P16. The suggestion submission form should confirm successful submission or provide an error message within 3 seconds of submission
P17. User searching should take no longer than 2 seconds
P18. Profile details should be fetched within 3 seconds.
P19. Log retrieval should be optimized for searches spanning up to 6 months.
P20. Log filtering should take under 2 seconds.
P21. Warnings should be delivered within 5 seconds via notification/email.
P22. Reports should be fetched within 3 seconds.
P23. Email generation should take under 5 seconds for reports under 1000 entries.

4.2 Safety and Security Requirements

The system must ensure secure handling of user and admin accounts through role-based access control (RBAC) and multi-factor authentication (MFA) for admins, with an optional MFA setting for users. AES-256 encryption will be used for stored financial data, and TLS 1.2 or higher will protect data transmissions, including transactions retrieved from bank apps. To prevent unauthorized access, the system will limit login attempts and lock accounts after multiple failed attempts. Users' financial data, including spending analysis and cost-cutting suggestions, will be kept private and not shared without consent, with compliance to GDPR and CCPA for data privacy. The group-sharing function will include privacy controls, allowing users to manage who can view their records. The system will monitor for suspicious transactions, alerting users of unusual spending behavior, and provide secure storage for scanned receipts to prevent tampering. Bank integration will use OAuth 2.0 for secure authentication without storing user banking credentials. To ensure system integrity, daily encrypted backups will be performed, allowing recovery within 24 hours in case of failure.

4.3 Software Quality Attributes

4.3.1 Usability

4.3.1.1 User-Friendly Interface

- The application should provide a clear user interface with a dashboard displaying spending summaries, categorised expenses, and financial insights that adapt to the user's needs.
- The application should use clear and simple language to guide users through the application, not the technical financial words.
- The application should use icons and colour-coded labels to improve navigation. For example, anything related to confirming the action will use green while denying action uses red.
- The colour in the application should follow the 6:3:1 rule to make the colour in the application more balanced and less eyes strained.

4.3.1.2 Accessibility

- The application should follow the Web Content Accessibility Guidelines (WCAG) 2.1 to ensure the usability for users with disabilities.
- The application should be compatible with the screen reader. For example, the image will provide the alternative text, the icon will come with descriptive text.

4.3.2 Reliability

4.3.2.1 Data Integrity

- The system should keep the logs of transactions to prevent the inconsistencies in the records.
- The system should support a rollback mechanism in order to restore to the previous state in case of failed transactions. By using the ROLLBACK function in the SQL.

4.3.2.2 Data Integrity & Fault Tolerance

- The system should maintain a 99.9% uptime with automatic failover. By using the Hadoop framework the system automatically changes to another server in case the current server fails.
- The system should perform daily automated backups to cloud storage.

4.3.3 Correctness

4.3.3.1 OCR Accuracy

- The application should achieve a minimum 95% accuracy for extracting information from the receipts under standard light conditions (around 400 LUX) with enough machine training.
- The extracted data should be validated against a commonly used format, for example, YYYY-MM-DD for dates.

4.3.3.2 Error Handling

- When OCR confidence falls below 90%, the application shall prompt the user to retry.
- The application should notify the user if the image is blurry, low-light, incomplete receipts.

5 Requirement Traceability

System Requirements			
Functional Requirement	Quality Requirement	Use Case	Use Case Description
FR001: The system shall allow the user to select between "bill splitting" or "bill customization."	QR001: The system shall respond to selection inputs within 1 second.	UC005	UD002
FR002: The system shall allow the user to edit or remove specific bill items after scanning.	QR002: The system shall process item selection and updates within 3 seconds.	UC005	UC003
FR001: The system shall allow the user to select between "bill splitting" or "bill customization."	QR001: The system shall respond to selection inputs within 1 second.	UC005	UD002
FR003: The system shall allow the user to select between "splitting" (Divides the total bill amount equally among a specified number of people) or "custom amount" (Allows the user to input a custom value for the amount).	QR003: The system shall complete bill splitting within 2 seconds for any valid input (i.e., total bill amount and number of people). QR004: The system shall validate that the custom amount input does not exceed the total bill amount before proceeding with the transaction.	UC005	UD005
FR004: The system shall automatically extract items, store name, payment method, date, and time from different types of bills.	QR005: The system shall complete data extraction within 3 seconds.	UC005	UD005
FR005: The system shall automatically categorize expenses into predefined categories (e.g., Groceries, Utilities, Rent, Entertainment) based on receipt details.	QR006: The system shall complete data categorization within 1 second.	UC005	UD005

FR005: The system shall automatically categorize expenses into predefined categories (e.g., Groceries, Utilities, Rent, Entertainment) based on receipt details.	QR006: The system shall complete data categorization within 1 second.	UC005	UD005
FR006: The system shall allow the user to search for cost-cutting alternatives on the web relevant to the user's interests.	QR007: The system shall complete web scraping within 10 seconds, with results ranked by relevance.	UC007	UD007
FR007: The system shall provide suggestions and comparisons for lower-cost alternatives.	QR008: The system shall update cost-saving suggestions at least once per day, reflecting real-time availability.	UC007	UD007
FR008: The system shall provide notifications for new suggestions.	QR009: The system shall deliver notifications within 5 seconds of new cost-cutting suggestions being found.	UC007	UD007
FR005: The system shall automatically categorize expenses into predefined categories (e.g., Groceries, Utilities, Rent, Entertainment) based on receipt details.	QR006: The system shall complete data categorization within 1 second.	UC005	UD005
FR009: The system shall display the total expense of the month.	QR010: The system shall load the dashboard expense summary within 2 seconds for optimal user experience.	UC005	UD005
FR010: The system shall display the expense trend over a selected time period (weekly/monthly).	QR011: The system shall complete trend calculations within 3 seconds for datasets under 12 months.	UC005	UD005
FR011: The system shall allow the user to sort displayed receipts (e.g., by time, by amount).	QR012: The system shall complete sorting operations within 1 second for datasets under 1000 receipts.	UC005	UD005
FR005: The system shall automatically categorize expenses into predefined categories (e.g., Groceries, Utilities, Rent, Entertainment) based on receipt details.	QR006: The system shall complete data categorization within 1 second.	UC005	UD005
FR012: The system shall allow users to integrate with bank APIs to sync transactions.	QR013: The system shall sync bank transactions within 5 seconds of new transactions being detected. QR014: The system shall encrypt all stored receipts using AES-256 to ensure data confidentiality.	UC006	UD006

FR013: The system shall allow the user to set and edit customizable budget periods.	QR015: The system shall persist budget settings and reflect changes immediately.	UC008	UD008
FR014: The system shall allow the user to set and edit a budget amount.	QR016: The system shall prevent negative or invalid values in budget input.	UC008	UD008
FR015: The system shall allow the user to track budget progress.	QR017: The system shall calculate and update budget progress when user inputs new receipts.	UC008	UD008
FR016: The system shall store scanned receipt images along with extracted data in categories.	QR014: The system shall encrypt all stored receipts using AES-256 to ensure data confidentiality.	UC005	UD005
FR017: The system shall allow the user to filter receipts by category.	QR018: Querying operations shall be completed in under 2 seconds.	UC005	UD005
FR018: The system shall allow the user to manually delete records.	QR014: The system shall encrypt all stored receipts using AES-256 to ensure data confidentiality.	UC005	UD005
FR019: The system shall generate reports for timeframes of 1 month, 6 months, and 1 year.	QR019: The system shall complete report generation within: 5 seconds for a 1-month timeframe, 10 seconds for a 6-month timeframe, 15 seconds for a 1-year timeframe	UC009, UC010	UD009, UD010
FR020: The system shall allow the user to manually generate reports.	QR020: The system shall generate manual reports within 10 seconds of being prompted.	UC009	UD009
FR021: The system shall allow the user to automatically generate reports.	QR021: The system shall generate automated reports within 1 minute of the scheduled time.	UC010	UD010
FR022: The system shall send the report to the user's email.	QR022: The system shall deliver emails within 30 seconds of report generation.	UC009, UC010	UD090, UD010
FR023: The system shall allow the user to upload receipts via camera capture.	QR023: The system shall detect blurry images and prompt the user to retake the photo before processing.	UC003	UD003

FR024: The system shall allow the user to upload receipts via file upload.	QR024: The system shall process uploaded receipt files within 3 seconds, with a maximum file size of 50MB. QR025: The system shall accept only PDF, JPEG, and PNG file types.	UC004	UD004
FR025: The system shall process uploaded receipts using OCR.	QR017: The system shall process OCR operations within 5 seconds maximum.	UC003, UC004	UD003, UD004
FR026: The system shall allow the user to create a group.	QR026: The system shall support up to 10 members per group.	UC011	UD011
FR027: The system shall allow the user to share an invitation link to join a group.	QR027: The system shall ensure that invitation links are valid for 24 hours and regenerated regularly.	UC011, UC012	UD011, UD012
FR028: The system shall allow the user to log shared expenses in a group.	QR028: The system shall sync group changes in real-time across all user devices.	UC014	UD014
FR029: The system shall allow the user to leave a group.	QR029: The system shall immediately remove access when a user leaves a group but retain past records.	UC013	UD013
FR030: The system shall provide notifications when new records are added by other users.	QR030: The system shall maintain detailed activity logs to track changes made by group members.	UC014	UD014
FR031: The system shall allow the user to log in using existing credentials.	QR031: The login process should take no longer than 2 seconds.	UC001	UD001
FR032: The system shall allow the user to register as a new user.	QR032: The registration form should have clear instructions and input validation to ensure users provide correct information.	UC002	UD002
FR033: The system shall allow the user to create and submit general suggestions.	QR033: The suggestion submission form should confirm successful submission or provide an error message within 3 seconds of submission.	UC015	UD015
FR034: The system shall allow the user to create and submit bug	QR034: The bug reporting feature should be available	UC015	UD015

reports.	24/7 with 99.9% per year uptime.		
FR035: The system shall allow the user to report specific users.	QR035: Reports about users must be anonymous to the public.	UC015	UD015
FR036: The system shall allow the user to search for other users.	QR036: User searching should take no longer than 2 seconds.	UC017	UD017
FR037: The system shall allow the user to view another user's profile, information, and activities.	QR037: Profile details should be fetched within 3 seconds.	UC017	UD017
FR038: The system shall allow the admin to fetch all logs.	QR038: Log retrieval should be optimized for searches spanning up to 6 months.	UC018	UD018
FR039: The system shall allow the admin to view all logs of the application.	QR039: The system shall display real-time system logs.	UC018	UD018
FR040: The system shall allow the admin to select the type of logs to view.	QR040: Log filtering should take under 2 seconds.	UC018	UD018
FR041: The system shall allow the admin to fetch user information.	QR041: The system shall ensure real-time synchronization of user management with the server.	UC017	UD017
FR042: The system shall allow the admin to send warnings to a user.	QR042: Warnings should be delivered within 5 seconds via notification/email.	UC019	UD019
FR043: The system shall allow the admin to suspend a user.	QR043: Suspension should immediately restrict login access.	UC020	UD020
FR044: The system shall allow the admin to ban a user.	QR044: Banning should permanently disable all user access and retain some data for compliance.	UC021	UD021
FR045: The system shall allow the admin to fetch user suggestions, bug reports, or user reports.	QR045: Reports should be fetched within 3 seconds.	UC022, UC023, UC024	UD022, UD023, UD024

FR046: The system shall allow the admin to view user suggestions, bug reports, or user reports.	QR046: Displaying reports should allow sorting by date or category.	UC022, UC023, UC024	UD022, UD023, UD024
FR047: The system shall allow the admin to delete user suggestions, bug reports, or user reports.	QR047: Deletion should be permanent and require admin confirmation.	UC022, UC023, UC024	UD022, UD023, UD024
FR048: The system shall allow the admin to forward user suggestions, bug reports, or user reports to the related department.	QR048: Forwarding should automatically generate a tracking ID for follow-up.	UC022, UC023, UC024	UD022, UD023, UD024
FR049: The system shall automatically generate email reports.	QR049: Email generation should take under 5 seconds for reports under 1000 entries.	UC022, UC023, UC024	UD022, UD023, UD024
FR031: The system shall allow the admin to log in using existing credentials.	QR031: The login process should take no longer than 2 seconds.	UC016	UD016

Requirement Traceability Sheet Original File:
https://docs.google.com/spreadsheets/d/1g6-TCD6_uJHf1Wn2zgQoujm4KaemlxcxZQzLZg4x3is/edit?usp=sharing

User Stories

Appendix A – Data Dictionary

Data Element	Description	Type	Size	Constraints
UserID	Unique identifier for each user.	Integer	10	Primary Key, Auto-increment
Username	User's login name.	String	50	Unique, Not Null
Password	User's password (hashed and salted).	String	255	Not Null
Email	User's email address.	String	100	Unique, Not Null
ReceiptID	Unique identifier for each receipt.	Integer	10	Primary Key, Auto-increment
ReceiptImage	Path to the stored receipt image.	String	255	
StoreName	Name of the store where the receipt was generated.	String	100	
PaymentMethod	Method of payment used (e.g., Credit Card, Cash).	String	50	
Date	Date of the transaction.	Date		
Time	Time of the transaction.	Time		
CategoryID	Unique identifier for each expense category (e.g., Groceries, Utilities).	Integer	10	Primary Key, Auto-increment
CategoryName	Name of the expense category.	String	50	Unique, Not Null
ExpenseAmount	Amount of the expense.	Decimal		
ItemName	Name of a specific item purchased on the receipt.	String	100	
ItemPrice	Price of a specific item purchased on the receipt.	Decimal		
Quantity	Quantity of a specific item purchased on the receipt.	Integer		
TaxAmount	Tax amount charged on the receipt.	Decimal		
DiscountAmount	Discount amount applied to the receipt.	Decimal		
TotalAmount	Total amount paid on the receipt.	Decimal		
TransactionID	Unique identifier for each bank transaction.	String	50	Unique, External Key from Bank API
TransactionType	Type of transaction (e.g., Debit, Credit).	String	20	
TransactionDescription	Description of the transaction.	String	255	
BudgetID	Unique identifier for each budget.	Integer	10	Primary Key, Auto-increment
BudgetName	Name of the budget.	String	50	Not Null
BudgetAmount	Amount allocated for the budget.	Decimal		Not Null
BudgetPeriod	Period for the budget (e.g., Weekly, Monthly).	String	20	
GroupID	Unique identifier for each group.	Integer	10	Primary Key, Auto-increment

GroupName	Name of the group.	String	50	Not Null
GroupDescription	Description of the group.	String	255	
InvitationLink	Unique link for inviting users to the group.	String	255	
SuggestionID	Unique identifier for each cost-cutting suggestion.	Integer	10	Primary Key, Auto-increment
SuggestionDescription	Description of the cost-cutting suggestion.	String	255	
SuggestionSource	Source of the suggestion (e.g., Web Service, Internal Algorithm).	String	50	
ReportID	Unique identifier for each report.	Integer	10	Primary Key, Auto-increment
ReportType	Type of report (e.g., Monthly Expense, Budget Performance).	String	50	
ReportData	Data contained in the report (e.g., JSON, CSV).	Text		
ReportGeneratedDate	Date the report was generated.	DateTime		
LogID	Unique identifier for each log entry.	Integer	10	Primary Key, Auto-increment
LogType	Type of log entry (e.g., Error, Warning, Info).	String	20	
LogMessage	Message contained in the log entry.	Text		
LogTimestamp	Timestamp of the log entry.	DateTime		

Appendix B - Group Log

Date	Activity	Members Involved
27/1/2025	● First Group Meeting ● Project Proposal Slide Completion	*Everyone*
19/2/2025	● Requirement Traceability interviews ● Requirement Traceability sheet ● First draft of BPMN diagram	*Everyone*
25/2/2025	● Continue working on BPMN diagram ● Continue working on Requirement Traceability Sheet	*Everyone*
27/2/2025	● Use Case Synthesis on Requirement Traceability Sheet ● Beginning of work on SRS report	*Everyone*
1/3/2025	● Completion of BPMN diagram ● Completion of Requirement Traceability sheet ● Continue working on SRS report	*Everyone*
2/3/2025	● Continue working on SRS report	*Everyone*
3/3/2025	● Completion of SRS report	*Everyone*